Resource Link: Python Program to Add Two Matrices - GeeksforGeeks

Random Forest Algorithm in Machine Learning - GeeksforGeeks

Machine Learning Random Forest Algorithm - Javatpoint

Linear Regression in Machine learning - GeeksforGeeks

Laboratory Work

1. **A simple machine-learning example in Python that demonstrates how to train a model to predict the species of iris flowers based on their sepal and petal measurements:**

```
# Load the necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
# Load the iris dataset
df = pd.read_csv('C:/Users/Acer/Desktop/Weka/iris_csv.csv')
# Split the data into features and labels
X = df[['sepallength', 'sepalwidth', 'petallength', 'petalwidth']]
y = df['class']
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Create an SVM model and train it
model = SVC()
model.fit(X_train, y_train)
# Evaluate the model on the test data
accuracy = model.score(X_test, y_test)
print('Test accuracy:', accuracy)
```

2. Python program using NumPy

```
import numpy as np
# Creating two arrays of rank 2
x = np.array([[1, 2], [3, 4]])
y = np.array([[5, 6], [7, 8]])
# Creating two arrays of rank 1
v = np.array([9, 10])
w = np.array([11, 12])
# Inner product of vectors
print(np.dot(v, w), "\n")
# Matrix and Vector product
print(np.dot(x, v), "\n")
# Matrix and matrix product
print(np.dot(x, y))
```

3. Python script using Scipy for image manipulation

```
import scipy
import imageio
from imageio import imread, imsave
from scipy.misc import imread, imsave, imresize
# Read a JPEG image into a numpy array
img = imread('D:/Programs / cat.jpg') # path of the image
print(img.dtype, img.shape)
# Tinting the image
img_tint = img * [1, 0.45, 0.3]
# Saving the tinted image
imsave('D:/Programs / cat_tinted.jpg', img_tint)
# Resizing the tinted image to be 300 x 300 pixels
```

```
img_tint_resize = imresize(img_tint, (300, 300))
# Saving the resized tinted image
imsave('D:/Programs / cat_tinted_resized.jpg', img_tint_resize)
```

4. Python script using Scikit-learn

```
# For Decision Tree Classifier
# Sample Decision Tree Classifier
from sklearn import datasets
from sklearn import metrics
from sklearn.tree import DecisionTreeClassifier
# load the iris datasets
dataset = datasets.load_iris()
# fit a CART model to the data
model = DecisionTreeClassifier()
model.fit(dataset.data, dataset.target)
print(model)
# make predictions
expected = dataset.target
predicted = model.predict(dataset.data)
# summarize the fit of the model
print(metrics.classification_report(expected, predicted))
print(metrics.confusion_matrix(expected, predicted))
```

5. Python program using Pandas for arranging a given set of data into a table

```
# importing pandas as pd
import pandas as pd
data = {"country": ["Brazil", "Russia", "India", "China", "South Africa"],
"capital": ["Brasilia", "Moscow", "New Delhi", "Beijing", "Pretoria"],
"area": [8.516, 17.10, 3.286, 9.597, 1.221],
```

```
"population": [200.4, 143.5, 1252, 1357, 52.98]
}
data_table = pd.DataFrame(data)
print(data_table)
```

6. Python program using Matplotlib for forming a linear plot

```
# importing the necessary packages and modules
import matplotlib.pyplot as plt
import numpy as np
# Prepare the data
x = np.linspace(0, 10, 100)
# Plot the data
plt.plot(x, x, label ='linear')
# Add a legend
plt.legend()
# Show the plot
plt.show()
```

7. Program to add two matrices using list comprehension

```
X = [[1,2,3],
    [4 ,5,6],
    [7 ,8,9]]
Y = [[9,8,7],
    [6,5,4],
    [3,2,1]]
result = [[X[i][j] + Y[i][j] for j in range
(len(X[0]))] for i in range(len(X))]
for r in result:
    print(r)
```

8. Program to demonstrate Supervised Learning Algorithms

```
import matplotlib.pyplot as plt
# Data for plotting
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
# Plotting a line
plt.plot(x, y, marker='o', linestyle='-', color='b', label='Line')
# Adding labels and title
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Simple Line Plot')
# Adding grid
plt.grid(True)
# Adding legend
plt.legend()
# Display the plot
plt.show()
```

9. Program to demonstrate Unsupervised Learning Algorithms

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
# Generate random data
data = np.random.rand(100, 2)  # 100 samples with 2 features
# Apply K-means clustering
kmeans = KMeans(n_clusters=3)  # 3 clusters
```

```
labels = kmeans.fit_predict(data)
# Plot the data points and the cluster centers
plt.scatter(data[:, 0], data[:, 1], c=labels, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
color='red', marker='X')
plt.show()
```

10. Program to demonstrate kmeans Elbow method

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn import metrics
from scipy.spatial.distance import cdist
import matplotlib.pyplot as plt
# Creating the data
x1 = np.array([3, 1, 1, 2, 1, 6, 6, 6, 5, 6,\
                    7, 8, 9, 8, 9, 9, 8, 4, 4, 5, 4])
x2 = np.array([5, 4, 5, 6, 5, 8, 6, 7, 6, 7, \
                    1, 2, 1, 2, 3, 2, 3, 9, 10, 9, 10])
X = np.array(list(zip(x1, x2))).reshape(len(x1), 2)
# Visualizing the data
plt.plot()
plt.xlim([0, 10])
plt.ylim([0, 10])
plt.title('Dataset')
plt.scatter(x1, x2)
plt.show()
```

11. Program to demonstrate kmeans clustering for different cluster value

```
import matplotlib.pyplot as plt
# Create a range of values for k
k_range = range(1, 5)
# Initialize an empty list to
# store the inertia values for each k
inertia_values = []
# Fit and plot the data for each k value
for k in k_range:
    kmeans = KMeans(n_clusters=k, \
                                    init='k-means++', random_state=42)
    y_kmeans = kmeans.fit_predict(X)
    inertia_values.append(kmeans.inertia_)
    plt.scatter(X[:, 0], X[:, 1], c=y_kmeans)
    plt.scatter(kmeans.cluster_centers_[:, 0],\
                            kmeans.cluster_centers_[:, 1], \
                            s=100, c='red')
    plt.title('K-means clustering (k={})'.format(k))
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.show()
# Plot the inertia values for each k
plt.plot(k_range, inertia_values, 'bo-')
plt.title('Elbow Method')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Inertia')
```

```
plt.show()
```

12. Python with the scikit-learn library to demonstrate the Elbow Method for a KNN

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
# Sample data
from sklearn.datasets import load_iris
data = load_iris()
X, y = data.data, data.target
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
# List to store accuracy scores
accuracy_scores = []
# Range of k values to try
k_values = range(1, 31)
# Train KNN for different k values and store the accuracy
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    accuracy_scores.append(accuracy)
# Plot the results
```

```
plt.figure(figsize=(10, 6))
plt.plot(k_values, accuracy_scores, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of Neighbors k')
plt.ylabel('Accuracy')
plt.xticks(k_values)
plt.grid()
plt.show()
```

13. Implement the K-Nearest Neighbors (KNN) algorithm to classify a dataset.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
        from sklearn.metrics import accuracy_score, confusion_matrix,
                                    classification_report
from sklearn.datasets import load_iris
# Step 1: Load the Iris dataset
data = load_iris()
X, y = data.data, data.target
# Step 2: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
# Step 3: Train the KNN model with a chosen value of k (e.g., k=5)
k = 5
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train, y_train)
```

```
# Step 4: Make predictions on the test set
y_pred = knn.predict(X_test)
# Step 5: Evaluate the model's performance
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
class_report = classification_report(y_test, y_pred)
print(f'Accuracy: {accuracy:.2f}')
print('Confusion Matrix:')
print(conf_matrix)
print('Classification Report:')
print(class_report)
# Step 6: Plot the confusion matrix
plt.figure(figsize=(8, 6))
plt.imshow(conf_matrix, interpolation='nearest', cmap=plt.cm.Blues)
plt.title('Confusion Matrix')
plt.colorbar()
tick_marks = np.arange(len(data.target_names))
plt.xticks(tick_marks, data.target_names, rotation=45)
plt.yticks(tick_marks, data.target_names)
# Normalize the confusion matrix
conf_matrix = conf_matrix.astype('float') / conf_matrix.sum(axis=1)[:,
np.newaxis]
thresh = conf_matrix.max() / 2.
for i, j in np.ndindex(conf_matrix.shape):
    plt.text(j, i, f'{conf_matrix[i, j]:.2f}', horizontalalignment='center',
             color='white' if conf_matrix[i, j] > thresh else 'black')
plt.ylabel('True label')
```

```
plt.xlabel('Predicted label')
plt.tight_layout()
plt.show()
```

14. Program to Build model using SVM with different kernels

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.datasets import load_iris
# Step 1: Load the Iris dataset
data = load_iris()
X, y = data.data, data.target
# Step 2: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
# Step 3: Train SVM models with different kernels
kernels = ['linear', 'poly', 'rbf']
svm_models = {}
for kernel in kernels:
    svm = SVC(kernel=kernel, degree=3, C=1.0, random_state=42)
    svm.fit(X_train, y_train)
    svm_models[kernel] = svm
# Step 4 and 5: Make predictions and evaluate each model
for kernel in kernels:
```

```
    svm = svm_models[kernel]
    y_pred = svm.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    conf_matrix = confusion_matrix(y_test, y_pred)
    class_report = classification_report(y_test, y_pred)
    print(f'Kernel: {kernel}')
    print(f'Accuracy: {accuracy:.2f}')
    print('Confusion Matrix:')
    print(conf_matrix)
    print('Classification Report:')
    print(class_report)
    # Step 6: Plot the confusion matrix
    plt.figure(figsize=(8, 6))
    plt.imshow(conf_matrix, interpolation='nearest', cmap=plt.cm.Blues)
    plt.title(f'Confusion Matrix - Kernel: {kernel}')
    plt.colorbar()
    tick_marks = np.arange(len(data.target_names))
    plt.xticks(tick_marks, data.target_names, rotation=45)
    plt.yticks(tick_marks, data.target_names)
    # Normalize the confusion matrix
    conf_matrix = conf_matrix.astype('float') / conf_matrix.sum(axis=1)[:,
np.newaxis]
    thresh = conf_matrix.max() / 2.
    for i, j in np.ndindex(conf_matrix.shape):
        plt.text(j, i, f'{conf_matrix[i, j]:.2f}', horizontalalignment='center',
                 color='white' if conf_matrix[i, j] > thresh else 'black')
    plt.ylabel('True label')
```

```
plt.xlabel('Predicted label')
plt.tight_layout()
plt.show()
```

15. Decision tree implementation

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.datasets import load_iris
# Step 1: Load the Iris dataset
data = load_iris()
X, y = data.data, data.target
# Step 2: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
# Step 3: Train the Decision Tree model
decision_tree = DecisionTreeClassifier(random_state=42)
decision_tree.fit(X_train, y_train)
# Step 4: Make predictions on the test set
y_pred = decision_tree.predict(X_test)
# Step 5: Evaluate the model's performance
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
class_report = classification_report(y_test, y_pred)
```

```
print(f'Accuracy: {accuracy:.2f}')
print('Confusion Matrix:')
print(conf_matrix)
print('Classification Report:')
print(class_report)
# Step 6: Plot the confusion matrix
plt.figure(figsize=(8, 6))
plt.imshow(conf_matrix, interpolation='nearest', cmap=plt.cm.Blues)
plt.title('Confusion Matrix')
plt.colorbar()
tick_marks = np.arange(len(data.target_names))
plt.xticks(tick_marks, data.target_names, rotation=45)
plt.yticks(tick_marks, data.target_names)
# Normalize the confusion matrix
conf_matrix = conf_matrix.astype('float') / conf_matrix.sum(axis=1)[:,
np.newaxis]
thresh = conf_matrix.max() / 2.
for i, j in np.ndindex(conf_matrix.shape):
    plt.text(j, i, f'{conf_matrix[i, j]:.2f}', horizontalalignment='center',
            color='white' if conf_matrix[i, j] > thresh else 'black')
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.tight_layout()
plt.show()
# Step 7: Plot the Decision Tree
plt.figure(figsize=(20,10))
```

```
plot_tree(decision_tree, feature_names=data.feature_names,
class_names=data.target_names, filled=True)
plt.title('Decision Tree')
plt.show()
```

16. Program to implement logistic regression

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.datasets import load_iris
# Step 1: Load the Iris dataset
data = load_iris()
X, y = data.data, data.target
# Step 2: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
# Step 3: Train the Logistic Regression model
log_reg = LogisticRegression(max_iter=200, random_state=42)
log_reg.fit(X_train, y_train)
# Step 4: Make predictions on the test set
y_pred = log_reg.predict(X_test)
# Step 5: Evaluate the model's performance
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
```

```
class_report = classification_report(y_test, y_pred)
print(f'Accuracy: {accuracy:.2f}')
print('Confusion Matrix:')
print(conf_matrix)
print('Classification Report:')
print(class_report)
# Step 6: Plot the confusion matrix
plt.figure(figsize=(8, 6))
plt.imshow(conf_matrix, interpolation='nearest', cmap=plt.cm.Blues)
plt.title('Confusion Matrix')
plt.colorbar()
tick_marks = np.arange(len(data.target_names))
plt.xticks(tick_marks, data.target_names, rotation=45)
plt.yticks(tick_marks, data.target_names)
# Normalize the confusion matrix
conf_matrix = conf_matrix.astype('float') / conf_matrix.sum(axis=1)[:,
np.newaxis]
thresh = conf_matrix.max() / 2.
for i, j in np.ndindex(conf_matrix.shape):
    plt.text(j, i, f'{conf_matrix[i, j]:.2f}', horizontalalignment='center',
            color='white' if conf_matrix[i, j] > thresh else 'black')
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.tight_layout()
plt.show()
```

17. Program to implement Naïve Bayes classifier for dataset

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.datasets import load_iris
# Step 1: Load the Iris dataset
data = load_iris()
X, y = data.data, data.target
# Step 2: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
# Step 3: Train the Naive Bayes model
naive_bayes = GaussianNB()
naive_bayes.fit(X_train, y_train)
# Step 4: Make predictions on the test set
y_pred = naive_bayes.predict(X_test)
# Step 5: Evaluate the model's performance
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
class_report = classification_report(y_test, y_pred)
print(f'Accuracy: {accuracy:.2f}')
print('Confusion Matrix:')
print(conf_matrix)
print('Classification Report:')
```

```
print(class_report)
# Step 6: Plot the confusion matrix
plt.figure(figsize=(8, 6))
plt.imshow(conf_matrix, interpolation='nearest', cmap=plt.cm.Blues)
plt.title('Confusion Matrix')
plt.colorbar()
tick_marks = np.arange(len(data.target_names))
plt.xticks(tick_marks, data.target_names, rotation=45)
plt.yticks(tick_marks, data.target_names)
# Normalize the confusion matrix
conf_matrix = conf_matrix.astype('float') / conf_matrix.sum(axis=1)[:,
np.newaxis]
thresh = conf_matrix.max() / 2.
for i, j in np.ndindex(conf_matrix.shape):
    plt.text(j, i, f'{conf_matrix[i, j]:.2f}', horizontalalignment='center',
            color='white' if conf_matrix[i, j] > thresh else 'black')
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.tight_layout()
plt.show()
```

18. Program to implement linear regression

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import load_iris
# Step 1: Load the Iris dataset
data = load_iris()
X, y = data.data, data.target
# Step 2: Prepare the data
# Let's predict the petal length (index 2) based on the other features
X = X[:, [0, 1, 3]]  # Use sepal length, sepal width, and petal width as features
y = data.data[:, 2]  # Use petal length as the target variable
# Step 3: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
# Step 4: Train the Linear Regression model
linear_reg = LinearRegression()
linear_reg.fit(X_train, y_train)
# Step 5: Make predictions on the test set
y_pred = linear_reg.predict(X_test)
# Step 6: Evaluate the model's performance
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f'Mean Squared Error: {mse:.2f}')
print(f'R^2 Score: {r2:.2f}')
```

```
# Step 7: Plot the results
plt.scatter(y_test, y_pred)
plt.xlabel('Actual Petal Length')
plt.ylabel('Predicted Petal Length')
plt.title('Actual vs Predicted Petal Length')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red')
plt.show()
```

Step-by-Step Implementation

1. **Import Libraries**: Import the necessary libraries for data handling, model training, and evaluation.
2. **Load Dataset**: Load the Iris dataset.
3. **Split Dataset**: Split the dataset into training and testing sets.
4. **Train Decision Tree Model**: Train the ……. model.
5. **Make Predictions**: Use the trained model to make predictions on the test set.
6. **Evaluate Model**: Evaluate the model's performance using appropriate metrics.
7. **Plot Results**: Optionally, visualize the results.

Explanation

- **Step 1**: Load the Iris dataset using load_iris from sklearn.datasets.
- **Step 2**: Split the dataset into training (70%) and testing (30%) sets using train_test_split.
- **Step 3**: Initialize and train the ……. classifier with the training data.
- **Step 4**: Make predictions on the test set using the trained model.
- **Step 5**: Evaluate the model using accuracy, confusion matrix, and classification report.
- **Step 6**: Plot the confusion matrix to visualize the classification performance.